LumaRPL

Programming Guide

RPL, Lua and CHIP-8 on the PicoCalc.



ClockworkPi PicoCalc | Raspberry Pi Pico 2 W

First edition | 0.10.0-rc.3 | November 2026

About this edition

LumaRPL Programming Guide - first edition, November 2026. Applies to firmware **0.10.0-rc.3**, based on the port 30.5 calculator and original evaluator.

Design direction and physical-device testing: **Iain Bennett**. The cover photograph was supplied by Iain Bennett. Screens inside the book are labelled production-UI host renders where used.

Procedures were checked against the maintained PicoCalc platform source and on-device reference. Runnable calculator/language examples have separate host verification records. A printed expected result is a check for the illustrated input, not a claim that every possible problem has a solution.

Guide source and build instructions accompany the repository under docs/manuals. Original explanatory text and build tooling follow the repository GPL-3.0 terms; embedded fonts and supplied artwork retain their separate terms.

Menu paths use >. Bold key names mean physical keys. A plus joins keys held together, as in Ctrl+T. RPL code uses « and » for program delimiters; Ctrl+B and Ctrl+E enter them. Mathematical answers may be printed in a simpler notation than the calculator's typeset display.

Save important work before trying an example that changes variables, settings or a document. Use the saving chapter in the User Guide and the editor chapter in the Programming Guide for the distinction between a calculator session and an SD document.

Contents

1 Choose a programming language 5

2 The shared source editor 7

3 Reference, examples and command insertion 9

4 Your first RPL program 10

5 Design a stack contract 12

6 Names, quotation and local variables 14

7 Decisions and repetition in RPL 15

8 Collections, units and calculator mathematics 17

9 Find and fix an RPL error 19

10 Your first Lua script 20

11 Lua libraries and the calculator bridge 22

12 Draw with Lua 24

13 Native plots from Lua 26

14 Keys, timing, sound and sprites in Lua 27

15 Lua errors and memory 29

16 Your first CHIP-8 assembly program 30

17 CHIP-8 registers, labels and sprites 31

18 Keypad and compatibility profiles 33

19 Debug assembly and use the sprite editor 35

20 Patterns for dependable small programs 37

A Selected RPL reference 39

B Lua API and example reference 42

C CHIP-8 instruction and example reference 56

D Practice answers 71

Calculator command additions 72

E Credits and further reading 73

F PDF font license notices 77

Index 79

1 Choose a programming language

LumaRPL gives you three ways to write a program. RPL works directly with calculator objects and the stack. Lua provides conventional variables, loops, text and drawing APIs. CHIP-8 assembly targets a small virtual machine with sprites, keys and timers.

Choose	When it is a good fit	Saved source
RPL	Exact calculator arithmetic, symbolic expressions, units and reusable stack operations	.48s under /db48x/data
Lua	Small scripts, animation, native graph callbacks and interactive tools	.lua under /db48x/lua
CHIP-8 assembly	Learning a small virtual machine and creating monochrome sprite programs	.asm under /db48x/chip8

A HOME RPL program is a calculator variable, saved with the session. An SD RPL file is an independent document. Lua and CHIP-8 use their own runtimes; a saved Lua variable or CHIP-8 register is not a HOME variable.

Read and run examples

RPL examples introduced as calculator entries are typed at the stack and followed by Enter. Source examples go into the appropriate editor. Use Ctrl+B for « and Ctrl+E for » on the PicoCalc keyboard.

The complete CHIP-8 examples explicitly state their compatibility profile. Instruction-reference fragments explain an operation; they are not necessarily complete visible programs. Lua canvas examples need screen.show(), and a key wait when you want to keep the picture visible during a running script.

Protect existing work

Save your calculator session before an example that stores names or changes settings. Save source with F1. Native documents and the calculator session have separate saving paths. Keep the calculator running after a failed save so you can retry without losing the working source.

The books' examples describe this LumaRPL build and keyboard. When adapting an HP RPL example, consult the current command reference for accepted names, syntax and supported objects.

2 The shared source editor

Press **Ctrl+P** and choose RPL files / editor, Lua scripts, or CHIP-8 / assembly. The file browser has F1 Open, F2 Run, F3 New, F4 Actions and F5 Back where those actions apply. Left/Right pages through files.

Create, save and reopen

1. Press F3 New and choose a new name, for example FIRST.
2. Type the source for the selected language.
3. F1 saves. F2 saves and runs. F4 opens Tools. F5 returns.
4. Reopen the file with F1 to continue editing it.

A successful RPL run returns its results to the calculator stack. Lua output can return to its editor after you leave the output view. Use the displayed controls after a CHIP-8 run. Save & run does not mean that the entire calculator session has also been saved.

Text controls

In a source editor	Action
Arrows	Move the caret
Enter	New line
Tab	Insert two spaces
Backspace	Delete before the caret, or the selection
Ctrl+A	Select all
Ctrl+K	Start or clear a selection anchor
Ctrl+C / Ctrl+X / Ctrl+V	Copy / cut / paste
Ctrl+Z / Ctrl+Y	Undo / redo
Ctrl+F / Ctrl+R	Find / replace
Ctrl+G	Go to line
Ctrl+U	Unindent
F4 Tools	Reference, examples, insertion, find, line navigation and history

The clipboard and individual edit operations are bounded; very large selections may need smaller operations. Keep a saved copy before a large rewrite. Source fonts use fixed cells so code columns stay aligned.

Unsaved changes

F5 or Esc from a dirty source offers Save, Discard and Cancel, defaulting to Cancel. If a selection is active, Esc clears it first. If saving fails, the source remains available. Undo history belongs to the live editing session; do not assume it is stored in a source file across a restart.

3 Reference, examples and command insertion

Language reference

In any of the three editors, open **F4 Tools > Language reference**. It opens the reference for that editor's language. Read a page, then use F5/Esc to return to the source.

Open example is a separate choice

F4 Tools > Open example is beside Language reference, not inside it. Choose an example and give it a new filename. LumaRPL preserves the parent source, caret and undo history while you explore the example.

Save the example if you want to keep it. Close it to return to the parent source. One nested example is allowed; if there is not enough memory to preserve the parent safely, LumaRPL refuses the operation rather than replacing your code.

A useful check is to type an unsaved comment in your own file, open and run an example, return, and Undo. The comment and its history should still belong to your original editor session.

Insert a command template

Tools > Insert command opens language-appropriate choices. The inserted template includes the punctuation or structure it needs and places the caret at the first useful field. Fill every required argument or placeholder before running it.

For a Lua drawing call, this saves typing the parentheses and argument separators. For an RPL control structure, it supplies the surrounding words. For assembly, it supplies an instruction form. It does not know your intended values or make an unfinished template a valid program.

Insertion is an edit: Undo can remove it, and opening the chooser or cancelling it leaves your current source intact.

4 Your first RPL program

RPL programs are sequences of operations between « and ». Entering the program object places it on the stack. EVAL executes its body.

Square a number

At the calculator, enter:

Calculator / RPL source

```
« DUP * » 'SQDEMO' STO
```

Then enter:

Calculator / RPL source

```
12 SQDEMO
```

The answer is **144**. SQDEMO is the stored name. DUP copies the input and multiplication combines the copies. Its stack contract is `x -> x^2`.

Save with Ctrl+S if you want the HOME name after a restart. In Programming > HOME programs, selecting SQDEMO and pressing F2 Run also executes it, using the current stack inputs.

The same program on SD

Create SQUARE in RPL files / editor and type:

Calculator / RPL source

```
«
  DUP *
»
```

Press F1 Save. Return to the calculator, put 12 on the stack, then select SQUARE in the RPL browser and press F2 Run. The result should again be 144. F2 Save & run in the editor executes the same program body.

The Run action executes the selected outer program. It does not recursively execute every program object that the body deliberately creates. A program can legitimately leave another quoted program on the stack.

Test the saved copy

Save the session for the HOME name. Save the SD file independently. Restart cleanly, reopen the SD source, and run it with a new input such as 5; expect 25. Loading the session and evaluating SQDEMO tests the HOME copy separately.

5 Design a stack contract

Write down which values a program consumes and what it leaves. RPL has no automatic local argument list unless you create one. Clear contracts make small programs reusable.

Two inputs: a hypotenuse

Program body:

Calculator / RPL source

```
«
  DUP *
  SWAP DUP *
  + SQRT
»
```

Contract: `a b -> sqrt(a^2+b^2)`. With 3 then 4 on the stack, the result is 5.

After squaring the top input b, SWAP brings a to the top, where DUP * squares it. Addition and SQRT finish the calculation. This works on suitable exact/calculator objects rather than converting everything to a Lua number first.

Useful stack operations

Command	Contract or purpose
DUP	x -> x x
SWAP	a b -> b a
DROP	Remove the top object
OVER	a b -> a b a
DEPTH	Push the current number of stack objects

To learn an unfamiliar roll command, try it on a short stack of distinct values and compare the result with the reference's contract. Do not infer a roll's direction from its name alone.

Comments and readable source

Use line breaks and indentation to separate stages. An RPL line comment starts with `@`, for example `@ Square the input` on its own line. Do not use Lua's `--` or assembly's semicolon comment marker in RPL. A short program with clear local names is often easier to maintain than a long chain of stack shuffles.

6 Names, quotation and local variables

Store a value or program

`42 'ANSWER' STO` stores a value. `'ANSWER' RCL` recalls it explicitly; evaluating ANSWER also retrieves its value. For a stored program, evaluating the unquoted name runs the program.

Quotation preserves an object for later use. `'X^2+1'` is an algebraic expression; `« DUP * »` is an RPL program; `'SQDEMO'` is a name. They are useful in different contexts.

Give inputs local names

Try this complete calculator entry:

Calculator / RPL source

```
3 4 « -> A B « A A * B B * + SQRT » » EVAL
```

It leaves **5**. The earlier value 3 binds A and the top value 4 binds B. The local structure consumes its inputs and gives the body named values without replacing HOME variables A and B.

Use local variables when a value is used repeatedly or when a formula's meaning is obscured by stack rearrangement. Keep the stack result contract visible even when the implementation uses local names.

Deliberately return a program

This example returns a program object rather than running it immediately:

Calculator / RPL source

```
« « 2 3 + » » EVAL
```

The remaining object is `« 2 3 + »`. A further EVAL gives 5. This ordinary quotation behaviour is preserved by the file/browser Run actions.

7 Decisions and repetition in RPL

Choose a branch

Calculator / RPL source

```
«
  IF 2 1 > THEN
    7
  ELSE
    9
  END
» EVAL
```

The comparison is true, so the result is **7**. The code before THEN leaves a condition. THEN and ELSE should leave compatible results when later code expects a particular stack shape. ELSE is optional.

A counted loop with a name

Calculator / RPL source

```
0 «
  1 5 FOR K
    K +
  NEXT
» EVAL
```

This adds 1+2+3+4+5 to the initial zero, leaving **15**. FOR names the counter; NEXT advances by one. START/NEXT repeats without a named counter. STEP uses an explicit step when you need a different increment.

Test before or after the body

Calculator / RPL source

```
0 «
  WHILE DUP 3 < REPEAT
    1 +
  END
» EVAL
```

The result is **3**. WHILE tests before the body, so the body can run zero times. DO/UNTIL tests after the body; a true UNTIL condition ends the loop.

Every loop needs an exit path. Keep the next iteration's required values on the stack. Press Esc to interrupt a long supported calculation instead of waiting for an accidental infinite loop to finish.

8 Collections, units and calculator mathematics

Lists and arrays

Lists use braces; arrays use square brackets. Try `{1 2 3} ListSum` for 6, or `[[1 2][3 4]] DET` for -2. GET/PUT access collection entries; positions start at 1.

Choose a list when it groups arbitrary objects. Choose a vector or matrix when array arithmetic expresses the problem. A list of arrays and one larger matrix are not interchangeable objects.

Preserve physical quantities

Calculator / RPL source

```
100_cm 1_m Convert
```

The result is one metre. Compatible dimensions are required; unit spellings are case-sensitive. Returning calculator objects from RPL preserves exactness, units and other calculator behaviour where the engine supports them.

Display settings inside examples

Angle and decimal settings can change a result's interpretation. `Rad` selects radians; `Deg` degrees. `3 Fix` changes decimal display, while Precision changes significant digits used in decimal work.

If a reusable program deliberately changes a setting, make that side effect part of its documented contract or restore the intended setting. The native workbenches' scoped evaluation does not mean that arbitrary user RPL programs have all their side effects rolled back.

Use the full reference

Ctrl+A opens the insertion catalogue at the calculator. Ctrl+K looks up command Help. In the editor, use Language reference or Insert command instead. The on-device reference covers more commands and settings than the selected stack-language reference in this guide.

9 Find and fix an RPL error

Parse errors and runtime errors

A parse error means the source could not be read as the intended object: a missing closing delimiter is one example. A runtime error happens after valid code begins executing: a missing argument or an invalid domain is another.

When an exact source location is available, the error action can take you to that line. Otherwise RPL offers Find rather than pretending it knows an exact location. Jumping to a location moves the caret; it does not rewrite your source or erase undo history.

A small diagnosis exercise

Create an SD program containing « + ». With no inputs, Run should report too few arguments. Supply two numbers on the stack and rerun it. If the result now appears, the problem was the stack contract, not the spelling of addition.

For an unmatched delimiter, correct the structure before rerunning. RPL source is parsed before a source-editor save/run path accepts it. Read the message rather than repeatedly pressing a function key on the calculator's settings menu.

Step through a program

The RPL editor's Tools route offers Debug. F1 Step executes the next operation; F2 Over steps over; F3 Run continues; F4 Vars opens variables; F5 Stop ends the stepping session. Watch how the stack changes after each operation. Debugging executes code; stored variables or other intentional side effects still matter.

Save source and important calculator state before debugging a program that changes names, files or settings. Use a short test input with a known answer. The hypotenuse program with 3 and 4 is a useful first stepping exercise.

10 Your first Lua script

Press **Ctrl+P > Lua scripts > F3 New**. Name it HELLO and enter:

Lua

```
print("Hello, LumaRPL")
print(2+3)
```

F2 saves and runs. The output is the greeting and 5. Leave the output view to return to editing. F1 saves without running.

Values and functions

Lua

```
local function square(x)
  return x*x
end
print(square(12))
```

This prints **144**. `local` keeps the name in its Lua scope. Lua's `*` multiplies values in an expression; it does not consume values from the calculator stack.

Repeat and choose

Lua

```
local total=0
for i=1,5 do
  total=total+i
end
if total==15 then
  print("Correct")
end
```

The output is **Correct**. Lua uses `then`, `end` and `do` in its own syntax; RPL's delimiters and uppercase structure words are a different language.

Use the Lua console

Ctrl+P > Lua console lets you try an expression directly. Enter `2+3` for 5, or use print for explicit output. Variables and the selected drawing-font profile persist in that Lua console session. Closing the console ends that session; it does not save variables into HOME or an SD file.

11 Lua libraries and the calculator bridge

Available libraries

The embedded runner opens base functions, table, string, math, utf8 and coroutine. Standard io, os, package and debug libraries are not opened, and dofile/loadfile are removed. You cannot assume that a desktop Lua script using require or arbitrary file access will run unchanged.

This is Lua 5.4.9 with LumaRPL extensions. Its allocation budget is 48 KiB and its displayed output is bounded. Keep tables/strings small and use data incrementally when possible.

Tables and strings

Lua

```
local samples={1,4,9,16}
for i,value in ipairs(samples) do
  print(i,value)
end
```

print separates multiple arguments with tabs. Tables are one-based by convention for these sequences. A very large generated table can exhaust the embedded allocation budget even if the source file is short.

Ask the calculator for one expression

Lua

```
print(calc.eval("2+3"))
```

This returns **text** describing 5. The bridge accepts a single algebraic expression, not a general sequence of RPL commands or a program with quotes. Its input is limited to 240 bytes and output is bounded.

`calc.eval("1/3")` can retain calculator exact arithmetic in the rendered answer. `tonumber` is only appropriate when the returned text is a plain Lua numeric

spelling; units, grouping or exact fractions need interpretation rather than blind conversion.

The bridge restores calculator stack depth and its temporary formatting override, and clears its reported error. Expressions resolve available calculator variables and use relevant calculator settings. It is not an unrestricted RPL execution API.

Degrees and radians

Lua math.sin and math.cos use radians. For a degree input, `math.sin(math.rad(90))` gives approximately 1. A Lua math result uses Lua numeric arithmetic; a native calculator expression can have different exact/precision behaviour.

12 Draw with Lua

The canvas is 320 by 320 pixels. The origin is upper-left: X grows right and Y grows down. Draw into the canvas, then call screen.show() to make the frame visible.

A first picture

Lua

```
screen.clear("black")
screen.text(10,10,"Hello, Iain!","cyan")
screen.line(10,35,200,35,"orange")
screen.circle(160,160,40,"green",false)
screen.show()
keyboard.wait()
```

A greeting, line and circle appear. A key wait keeps the running script's picture available until a key arrives; Esc interrupts.

Colours accept a supported name, a `#RRGGBB` string or an integer from 0 to 0xffffff. The display quantizes those requests to its 256-colour RGB332 palette. Similar requested colours can become the same displayed colour.

Use an appropriate font

New Lua sessions start with compact proportional text: 13-pixel text and 16-pixel line spacing. `screen.font("mono")` selects compact fixed cells for aligned tables. `screen.font("legacy")` selects the previous header-font behaviour for an existing script that needs it.

`screen.font()` reads the active profile. Selecting one returns the previous profile, allowing a script to restore it. The profile stays within the Lua session; it does not change calculator settings.

Measure before placing text

Lua

```
screen.clear("black")
local label="Centred"
local w,h=screen.measure(label)
screen.text(math.floor((320-w)/2),20,label)
screen.rect(8,18,304,h+4,"gray",false)
screen.show()
keyboard.wait()
```

Measurement and drawing use the same active font layout. Text outside the canvas is clipped. Multiline text uses the selected profile's layout; keep enough vertical space for every line.

A moving dot

Lua

```
for x=20,280,4 do
  screen.clear("black")
  screen.circle(x,160,6,"orange",true)
  screen.show()
  system.sleep(20)
end
```

Redraw a coherent frame rather than accumulating unwanted trails. Keep loops small and let Esc interrupt. The Lua API uses the application's existing framebuffer; it does not create another full-screen buffer for your script.

13 Native plots from Lua

Use the native plotting API for mathematical curves and surfaces. It gives your script the same native viewing controls as the calculator's graph tools.

Compare sine and cosine

Lua

```
plot.clear()
plot.window(-6.3,6.3,-2,2)
plot.function2d(function(x) return math.sin(x) end,
  {colour="cyan",label="Sine"})
plot.function2d(function(x) return math.cos(x) end,
  {colour="orange",label="Cosine"})
plot.show()
print("Returned from graph")
```

Up to three 2D curves can be registered. A callback returns a finite number, or nil for a gap. It should be a small mathematical function: avoid changing shared application state while the viewer samples it.

A saddle surface

Lua

```
plot.clear()
plot.surface(function(x,y) return x*x-y*y end,
  {xmin=-3,xmax=3,ymin=-3,ymax=3})
plot.show()
```

Arrows rotate, +/- zooms and F1 resets the camera in the surface viewer. Leaving plot.show() continues the Lua script. Optional camera angles are radians, with the documented pitch and zoom limits in the API reference.

Call plot.clear() before changing to a new plot definition. Plot windows and callbacks belong to the Lua session; the API does not add general parameter-management methods for native workbenches.

14 Keys, timing, sound and sprites in Lua

Read a key without blocking

keyboard.poll() returns a queued key name, or nil if none is available. keyboard.down(name) checks whether a key is held. keyboard.wait() waits; an optional positive timeout is in milliseconds. Esc remains interruption rather than an ordinary application key event.

Named keys include left, right, up, down, enter, space, backspace and f1-f5. Printable single characters can also be used. system.sleep(ms) delays cooperatively; system.millis() reads the millisecond counter. Use elapsed-time differences, not assumptions about an exact display frame rate.

Play a short tone

Lua

```
sound.beep(880,150)
sound.stop()
```

The tone frequency is in hertz and duration in milliseconds. Supported nonzero frequencies are 20..16000 Hz; duration is at most 10000 ms. A zero frequency stops sound. These bounds are part of the API, not musical tuning recommendations.

Draw a saved colour sprite

Create and save a 16 by 16 colour drawing in Sprite editor. In Lua:

Lua

```
screen.clear("black")
local ok,err=screen.sprite("DEMO.sprite",40,60)
if ok then
  screen.show()
  keyboard.wait()
else
  print(err)
end
```

Replace DEMO.sprite with your saved filename. Paths are restricted to the calculator's sprite directory. Transparency is respected and drawing clips to the canvas. Invalid/unavailable files return nil and an error message instead of a successful drawing.

The bundled SPRITE.lua demonstrates the format. A colour .sprite document is separate from a CHIP-8 one-bit mask and its assembly export.

15 Lua errors and memory

Find the actual line

A missing parenthesis is a syntax error. Calling a nil value or dividing through an unsuitable domain can be a runtime error. On an unhandled script error, the editor can offer a line jump when the runtime provides a location.

Correct the source and rerun. Undo history is retained during supported save/run recovery. If a script catches an error with pcall, it has handled that failure itself, so there may be no unhandled-error jump to perform.

Catch an expected failure

Lua

```
local ok,err=pcall(function()
  error("Try again")
end)
print(ok,type(err))
```

The output is false and string. Use pcall around a failure you can usefully recover from; do not hide every error and continue with invalid data.

Stay within the budget

The runner bounds Lua allocations to 48 KiB. Long output is bounded too. Reduce a table, avoid repeated string concatenation in a large loop, and release references you no longer need. Save source before closing an editor or restarting.

A program can be syntactically valid and still exceed memory or computation limits. Test a small input first. The built-in Esc hook supports interruption during Lua execution, waits and native plotting callbacks.

16 Your first CHIP-8 assembly program

CHIP-8 is a virtual machine, not the Pico 2 W's native Arm instruction set. Programs operate on registers and memory, draw monochrome sprites and read a sixteen-key keypad.

Draw the digit 5

Create FIRST in **Ctrl+P > CHIP-8 / assembly > F3 New**:

CHIP-8 assembly

```
CLS
LD V0, 5
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

Save it, return to the browser, and choose **F4 Actions > Compatibility profile > SUPER-CHIP modern**. Then press F2 Run. The digit 5 appears near the upper-left of the CHIP-8 display. Press and release X to satisfy the key wait and return.

EXIT is a SUPER-CHIP extension; using an incompatible profile changes whether the example can finish as written. The result is a picture, not the number 5 on the calculator stack.

Source and ROM are different files

Keep FIRST.asm for editing. Running assembles a separate .ch8 ROM. Opening a ROM does not reconstruct its original assembly source or labels. Browser actions select profile and other run options.

The supplied MOVE.asm and BEEP.asm are original local demos. No proprietary HP interpreter or calculator ROM is needed to use the virtual machine.

17 CHIP-8 registers, labels and sprites

Registers and numbers

V0 through VF are eight-bit registers. I is the memory index; PC is the instruction pointer. VF is also used for carry, borrow or drawing collision by instructions that specify it. Do not keep an unrelated long-lived value in VF across such an operation.

Assembler numbers use decimal or prefixes such as `0x` for hexadecimal. RPL's `#FFD2h` notation belongs to the calculator, not this assembler. Opcodes/register spellings are case-insensitive; label spelling is retained and labels are case-sensitive.

Labels and data

A label ends with a colon. Semicolon or # begins an assembly comment. DB emits bytes; DW emits words. Programs start at 0x200 in the VM's 4096-byte memory. Assembled program/data must fit the remaining 3584 bytes.

Draw your own eight-pixel-wide image

This complete SUPER-CHIP example draws a small box:

CHIP-8 assembly

```
CLS
LD I, box
LD V0, 12
LD V1, 8
DRW V0, V1, 5
LD V2, K
EXIT
box:
DB 0xF0, 0x90, 0x90, 0x90, 0xF0
```

Each byte gives one row, most significant bit first. DRW combines pixels using XOR. Drawing the same sprite twice at the same position erases it; VF reports a collision where a previously lit pixel was turned off.

Timers

The delay and sound timers count at 60 Hz. To make a roughly half-second sound, load 30 into a register, then copy it to ST. Poll DT in a loop if you need to wait for a timer. The timer is separate from the number of VM instructions executed per display update.

Use the reference's LD forms for fonts, BCD and memory. SAVE/LOAD can alter I differently by profile, so choose the profile expected by the source rather than adjusting a few instructions by guesswork.

18 Keypad and compatibility profiles

The sixteen-key mapping

PicoCalc keys	CHIP-8 hexadecimal keypad values
1 2 3 4	1 2 3 C
Q W E R	4 5 6 D
A S D F	7 8 9 E
Z X C V	A 0 B F

Press and release a mapped key for `LD Vx,K`. In the first program, X maps to zero. Esc leaves/interrupts the VM through the surrounding application; it is not one of the sixteen VM keys.

Select the expected interpreter behaviour

Profiles are COSMAC VIP, CHIP-48, SUPER-CHIP modern and SUPER-CHIP legacy. They select historical differences in shifts, memory transfers, offset jumps, draw timing and extensions. The profile matters even when two ROMs draw the same first frame.

Choose the profile in browser Actions. When importing an outside ROM, use its documented target. A profile error is a better diagnosis than editing a working instruction blindly. XO-CHIP and arbitrary native HP/Saturn binaries are outside this VM.

Display modes

Ordinary CHIP-8 uses 64 by 32 logical pixels. SUPER-CHIP can use 128 by 64. The PicoCalc renderer scales the virtual pixels; it does not reinterpret them as full-colour sprite documents.

HIGH/LOW, scrolling, large fonts, flags and EXIT depend on the supported profile. Read the instruction page's qualification before using an extension.

19 Debug assembly and use the sprite editor

A build error

If assembly fails, read its message and source line. The error action can move the source caret to that line without destroying the file or its undo history. Check operand count, label spelling and number range first.

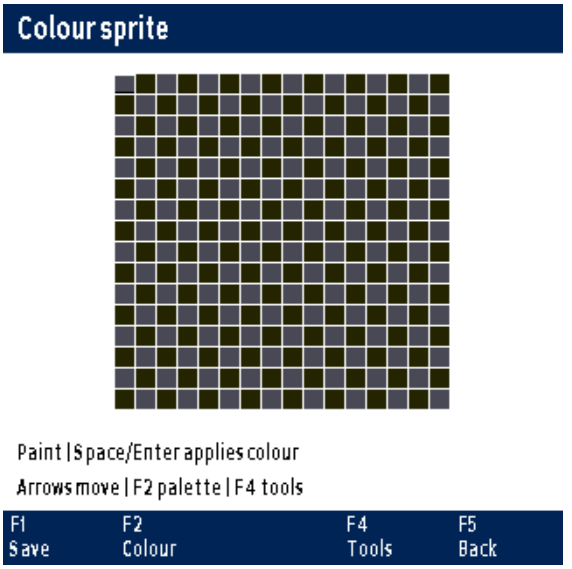
A runtime VM error can identify a program counter. Where its mapping is available, the editor maps that address back to source. Data directives and instructions have different sizes; an arbitrary PC is not simply the same as a source line number.

A blank screen

First run the complete digit example from chapter 16. If that works, compare your own I/font setup, coordinates, DRW height and key wait. Loading a number into V0 alone does not draw a number.

Paint a reusable sprite

Ctrl+P > Sprite editor opens a 16 by 16 grid. Arrows move; Space/Enter paints; Backspace erases. F2 Colour chooses an RGB332 paint colour or transparency. F1 saves. F4 Tools contains open, mode and export choices. F5 returns with unsaved-work protection.



Sprite editor, rendered by the production UI on the host.

Colour sprite mode saves a `.sprite` document under `/db48x/sprites/`, with 256 RGB332 pixel bytes and a separate transparency mask in a checked file. Lua screen.sprite() draws it with clipping and transparency.

CHIP-8 sprite mode remains one bit per pixel; selected colours affect its preview. Switching a populated colour drawing to monochrome asks first. Explicit mask export makes opaque pixels set bits and transparent pixels clear bits; the actual colour value does not determine the mask bit.

20 Patterns for dependable small programs

Start with an observable result

In RPL, choose a known stack answer. In Lua, print a value or draw a labelled picture. In CHIP-8, draw a digit and wait for a key. A successful dispatch with no observable result is a poor first test.

Keep work and execution separate

Save source before Run. Preserve a small test input. Check a result before storing it into HOME or replacing a working document. A script's working variables are not a durable SD save.

Prefer a native plot for mathematics

Lua screen drawing is suited to a custom canvas. plot.function2d and plot.surface use the native renderer, sampling and interaction. Keep callbacks small and return nil for gaps. A graph can take time without being frozen; Esc provides the supported cancellation path.

Treat boundaries as part of the design

Boundary	Practical habit
A formula field is at most 159 bytes	Keep a longer derivation in source or on the calculator stack
Lua allocations are bounded	Test small tables and short strings first
Output is bounded	Print summaries, not an unbounded trace
Assembled CHIP-8 code/data fits 3584 bytes	Count data as well as instructions
Undo/clipboard edits are bounded	Save before a large replacement
PSRAM is volatile	Save to SD before power-off

A three-language restart test

Save SQUARE.48s, HELLO.lua and FIRST.asm. Confirm each runs. Put an unsaved comment into a parent source, open an example, run it and return; confirm the parent and Undo still work. Save the parent before leaving. Return cleanly to the loader and reopen the three source files after restart. Test their results again.

A Selected RPL reference

This section is a working reference for the tutorials. It does not replace the full on-device command/settings reference. Inputs are written in the order you enter them; the last is the top of the stack. A parameter in parentheses is explanatory notation, not literal input.

Stack and names

Entry	Purpose	Example
DUP	Copy top	7 DUP * gives 49
SWAP	Exchange top two	10 3 SWAP - gives -7
DROP	Remove top	1 2 DROP leaves 1
OVER	Copy second	1 2 OVER leaves 1,2,1
STO	Store under quoted name	42 'ANSWER' STO
RCL	Explicit recall	'ANSWER' RCL
EVAL	Evaluate an object	« 2 3 + » EVAL gives 5

Objects and mathematics

Entry	Meaning or example
'X^2+1'	Quoted algebraic expression
« DUP * »	Quoted RPL program
{1 2 3}	List
[[1 2][3 4]]	Two-row matrix
(1;2)	Rectangular complex value
100_cm 1_m Convert	Convert centimetres to metres
#FFh Bin	Show hexadecimal FF in binary
255 Hex	LumaRPL direct ordinary-integer conversion
[[1 2][3 4]] DET	Determinant -2
{1 2 3} ListSum	Sum 6

Settings and structure

Entry	Effect
Rad / Deg / Grad	Select an angle convention
3 Fix	Three displayed decimal places
16 Precision	Sixteen significant decimal calculation digits
IF ... THEN ... ELSE ... END	Choose a branch
first last FOR K ... NEXT	Count with local K
first last START ... NEXT	Count without a named counter
WHILE ... REPEAT ... END	Test before each body
DO ... UNTIL ... END	Test after the body
-> A B « ... »	Bind the supplied inputs to local names

For a sixth soft-menu column use Ctrl+F5. For a long command name use the compact label or its full reference/catalogue spelling. A setting that needs a numeric argument is not a no-argument toggle.

B Lua API and example reference

The following entries describe the embedded API shipped with LumaRPL. Complete scripts are labelled Script; fragments are labelled Fragment. An API's optional-argument notation uses square brackets for explanation and is not typed literally into Lua.

Start here

Run your first script

Make a tiny script first, then run it from the editor. You can come back and change it as often as you like.

1. Press Ctrl+P and choose Lua scripts.
2. Press F3 New and name the script HELLO. Type the code below.
3. Press F2 Save & run. Read the output, then Esc returns to the editor.

Example script / program

```
print(2+3)
```

What to expect: Prints 5.

Try a quick calculation

Use the console when you want an answer without creating a file. Its variables are temporary.

1. Press Ctrl+P and choose Lua console.
2. Type the example and press Enter.
3. Press Esc at an empty prompt to return.

Example script / program

```
print(12*12)
```

What to expect: You should see 144.

Save and reopen a script

Save a script on SD when you want it after a restart. The console does not save your code as a file.

1. In Lua scripts, press F3 New and name it SAVETEST.

2. Type the example. F1 saves it; F5 returns to the file list.

3. Select SAVETEST.lua and press F1 Open to check it. F2 runs it.

Example script / program

```
print("Saved script")
```

What to expect: Running the saved script prints Saved script.

Write Lua

Reuse a calculation

A function lets you name a calculation and use it with different inputs. Here square takes a number and returns its square.

Example script / program

```
local function square(x)
  return x*x
end
print(square(12))
```

What to expect: Prints 144.

Repeat a task

Use a for loop to repeat a calculation. This one adds the numbers 1 through 5. The if line checks the answer before printing a message.

Example script / program

```
local total=0
for i=1,5 do total=total+i end
if total==15 then print("Correct") end
```

What to expect: Prints Correct.

Keep a list of values

Use a table for a list; the first item is number 1. Use .. to join pieces of text. Change the names below and run it again.

Example script / program

```
local names={"Iain","LumaRPL"}
print(names[1]..": "..names[2])
```

What to expect: Prints Iain: LumaRPL.

Calculate with Lua maths

Use math.sqrt for a square root. Lua sine and cosine take radians; math.rad converts degrees before the calculation. Calculator angle settings do not change Lua math functions.

Example script / program

```
print(math.sqrt(81))
print(math.sin(math.rad(90)))
```

What to expect: Prints 9.0 and 1.0 on separate lines.

print

```
print(value, ...)
```

Show an answer in the output reader. Each call starts a new line; multiple arguments are separated by tabs. For text placed on the drawing canvas, use screen.text instead.

Example script / program

```
print("Answer",6*7)
```

What to expect: Prints Answer followed by a tab and 42.

calc.eval

```
answer = calc.eval(expression)
```

Ask the calculator to evaluate an expression and return the answer as text. Write the expression in calculator syntax, without surrounding single quotes. Calculator angle and precision settings apply. Your stack is kept as it was.

Example script / program

```
print(calc.eval("2+3"))
```

What to expect: Prints 5. It returns text, so use tonumber only when a plain Lua number is suitable.

Draw on screen

Draw your first picture

Clear the canvas, add your message, then show the frame. You can change the position and colour and run it again.

1. Create a Lua script named GREETING.

2. Type the three lines below and press F2 Save & run.

Example script / program

```
screen.clear("black")
screen.text(10,10,"Hello!","cyan")
screen.show()
```

What to expect: You should see Hello! near the upper-left corner. Esc returns to your script.

Choose positions and colours

The upper-left pixel is (0,0). Increase x to move right and y to move down. Choose a named colour, or use a value such as "#FF8000". The screen uses the nearest available RGB332 colour.

Example script / program

```
screen.clear("black")
screen.pixel(20,20,"yellow")
screen.show()
```

What to expect: Shows a yellow pixel on black. Available names: black, white, red, green, blue, cyan, magenta, yellow, orange, gray.

screen.clear

```
screen.clear([colour])
```

Start a fresh drawing by filling the canvas. Omit the colour for black. Draw your objects next, then call screen.show to display the frame.

Example script / program

```
screen.clear("blue")
screen.show()
```

What to expect: Shows a blue canvas.

screen.pixel

```
screen.pixel(x, y [,colour])
```

Draw one pixel at a chosen position. Omit the colour for white. Positions outside the canvas are clipped.

Example script / program

```
screen.clear()
screen.pixel(100,100,"red")
screen.show()
```

What to expect: Shows one red pixel at (100,100).

screen.line

```
screen.line(x1, y1, x2, y2 [,colour])
```

Draw a straight line between two positions. The example joins (20,20) to (280,280).

Example script / program

```
screen.clear()
screen.line(20,20,280,280,"cyan")
screen.show()
```

What to expect: Shows a cyan diagonal.

screen.rect

```
screen.rect(x, y, width, height [,colour [,filled]])
```

Draw a rectangle by giving its top-left position, width and height. Add false for an outline; leave it out for a filled rectangle.

Example script / program

```
screen.clear()
screen.rect(20,30,120,60,"green",false)
screen.show()
```

What to expect: Shows a green rectangle outline.

screen.circle

```
screen.circle(x, y, radius [,colour [,filled]])
```

Draw a circle around a centre point. Give its radius, and add true if you want it filled. Without that final argument it draws an outline.

Example script / program

```
screen.clear()
screen.circle(160,160,50,"orange",true)
screen.show()
```

What to expect: Shows a filled orange circle.

screen.text

```
screen.text(x, y, text [,colour])
```

Place a message on the canvas. Give its top-left position and the text in quotes. Add a colour if you want one; otherwise the text is white. Use \n inside the string for a new line.

Example script / program

```
screen.clear()
screen.text(10,10,"Hello, Iain!","white")
screen.show()
```

What to expect: Shows the greeting near the upper-left corner.

screen.font

```
screen.font() or screen.font("compact" | "mono" | "legacy")
```

Choose compact for ordinary text or mono for aligned columns. Use legacy when an older script needs its previous font. With no argument, screen.font() tells you

the current choice. Changing it returns the previous choice and lasts until this Lua session closes.

Example script / program

```
print(screen.font())
screen.font("mono")
print(screen.font())
```

What to expect: Prints compact, then mono in a new session.

New sessions start with compact. The choice does not change the calculator stack font.

screen.measure

```
width, height = screen.measure(text)
```

Measure text before placing it. The two returned values are its width and height in pixels, using the active font. Select the font first so measurement and drawing agree.

Example script / program

```
local w,h=screen.measure("Hello")
print(w>0,h>=13)
```

What to expect: Prints true and true. Width depends on the selected font.

screen.show

```
screen.show()
```

Display the frame you have drawn. Make all the changes for one frame, then show it once. When a drawing script finishes, Esc leaves its canvas.

Example script / program

```
screen.clear()
screen.text(10,10,"One frame")
screen.show()
```

What to expect: Shows One frame.

screen.sprite

```
ok, error = screen.sprite(filename, x, y)
```

Draw a colour sprite you saved in Sprite editor. Give the .sprite filename and a position. Transparent pixels leave the canvas visible. If the file cannot be read, the function returns nil and an error you can print.

Example script / program

```
local ok,err=screen.sprite("DEMO.sprite",20,30)
if ok then screen.show() else print(err) end
```

What to expect: Draws DEMO.sprite if present; otherwise prints a readable error. Replace the name with your saved sprite.

Save sprites under /db48x/sprites. Use a filename from that folder, not a path to another directory.

Plot graphs

Compare two curves

Give each curve its own function and label. Lua trigonometric functions use radians.

1. Save this as a Lua script, then press F2 Save & run.

Example script / program

```
plot.clear()
plot.window(-4,4,-2,2)
plot.function2d(function(x) return math.sin(x) end,
  {colour="cyan",label="Sine"})
plot.function2d(function(x) return math.cos(x) end,
  {colour="orange",label="Cosine"})
plot.show()
```

What to expect: The native viewer shows sine and cosine together. Use its labelled graph controls; Esc returns.

plot.clear

```
plot.clear()
```

Start a new Lua plot. This removes the Lua curves you previously added and restores the default window. It leaves saved calculator graphs alone.

Example script / program

```
plot.clear()
plot.function2d(function(x) return x*x end)
plot.show()
```

What to expect: Opens the native view with one parabola. Esc returns to Lua.

plot.window

```
plot.window(xmin, xmax, ymin, ymax)
```

Choose the visible x and y ranges for a 2D plot: xmin, xmax, ymin, ymax. Each minimum must be smaller than its maximum. Add a curve, then show the plot.

Example script / program

```
plot.clear()
plot.window(-4,4,-1,16)
plot.function2d(function(x) return x*x end)
plot.show()
```

What to expect: Shows the parabola in the selected window.

plot.function2d

```
plot.function2d(function, [options])
```

Add a curve to a 2D plot. Your function receives x and returns y; return nil to leave a gap. You can add up to three curves. The optional table chooses the colour and a short label.

Example script / program

```
plot.clear()
plot.window(-4,4,-2,2)
plot.function2d(function(x) return math.sin(x) end,
  {colour="cyan",label="Sine"})
plot.show()
```

What to expect: Shows a cyan sine curve. Lua math.sin uses radians.

Labels use printable ASCII and at most 79 bytes. Keep drawing and plot changes outside the sampling function.

plot.surface

```
plot.surface(function, [options])
```

Make a 3D surface from a function of x and y. Return the height z at each position, or nil for a gap. The options below choose the horizontal bounds.

Example script / program

```
plot.clear()
plot.surface(function(x,y) return x*x-y*y end,
  {xmin=-2,xmax=2,ymin=-2,ymax=2})
plot.show()
```

What to expect: Opens a saddle surface in the native 3D viewer.

Optional camera settings: yaw, pitch and zoom. Angles are radians; pitch is -1.4 to 1.4 and zoom is 0.4 to 2.5.

plot.show

```
plot.show()
```

Open the native graph viewer with the curves or surface you added. Use its labelled controls to explore. Returning to Lua continues the script after plot.show().

Example script / program

```
plot.clear()
plot.function2d(function(x) return x end)
plot.show()
print("Back from plot")
```

What to expect: Shows y=x, then prints Back from plot when you leave the viewer.

Use keys and sound

keyboard.poll

```
key = keyboard.poll()
```

Check for a key without waiting. It returns a key name, or nil when no key is queued. Arrow keys return names such as "left"; ordinary letter keys return a single character. Esc interrupts the script.

Example script / program

```
local key=keyboard.poll()
print(key or "No key")
```

What to expect: Prints a queued key, or No key.

keyboard.wait

```
key = keyboard.wait([timeout_ms])
```

Wait for a key, optionally with a timeout in milliseconds. With no timeout it waits until a key arrives. A timed-out wait returns nil. Esc cancels the script.

Example script / program

```
local key=keyboard.wait(1000)
print(key or "Timed out")
```

What to expect: Waits up to one second, then prints a key or Timed out.

keyboard.down

```
held = keyboard.down(key)
```

Check whether a key is currently held. Give an arrow name such as "left", f1 through f5, or one printable character. The answer is true or false.

Example script / program

```
print(keyboard.down("left"))
```

What to expect: Prints true while Left is held, otherwise false.

system.sleep

```
system.sleep(milliseconds)
```

Pause for a number of milliseconds. For example, 100 means a tenth of a second. Esc can still interrupt the pause.

Example script / program

```
print("Wait")
system.sleep(100)
print("Done")
```

What to expect: Prints Wait, pauses about a tenth of a second, then prints Done.

system.millis

```
milliseconds = system.millis()
```

Measure how long something takes. Save the counter before an operation, then subtract it from the counter afterward. This counter measures elapsed milliseconds, not the date or clock time.

Example script / program

```
local start=system.millis()
system.sleep(20)
print(system.millis()-start>=20)
```

What to expect: Prints true.

sound.beep

```
sound.beep(frequency_hz, duration_ms)
```

Play a tone by giving its frequency in hertz and its length in milliseconds. The example makes a short, high note.

Example script / program

```
sound.beep(880,100)
```

What to expect: Plays a short 880 Hz tone when sound is available.

Use 20 to 16000 Hz and up to 10000 ms. A frequency of 0 is silence.

sound.stop

```
sound.stop()
```

Turn off the current tone. You can use this when leaving a sound-producing part of your script.

Example script / program

```
sound.beep(440,50)
sound.stop()
```

What to expect: Plays a short tone and stops sound.

Solve a problem

Use the Lua libraries

Use math for numerical functions, string for text, and table for lists. utf8 and coroutine are also available. Files are opened and saved through the calculator editor; desktop examples using io, os, require or loadfile will not work here.

Example script / program

```
print(type(math),type(io),type(os),type(package))
```

What to expect: Prints table, nil, nil, nil separated by tabs.

Find an error in a script

Read the line number in the error message, return to the editor and check that line. A missing quote or end is often the cause. When you expect a function to fail, pcall lets you handle the error and continue.

Example script / program

```
local ok,err=pcall(function() error("Try again") end)
print(ok,type(err))
```

What to expect: Prints false and string. The script continues.

A script runs out of memory

If a script reports a memory error, save the source and make its tables or strings smaller. Keep only the values you need. This small list is a useful starting point; it stores ten squares.

Example script / program

```
local values={}
for i=1,10 do values[i]=i*i end
print(values[10])
```

What to expect: Prints 100. Start with small tables before increasing their size.

The Lua allocation budget is 48 KiB. Save before closing the editor; console variables disappear when you close the console.

A drawing does not appear

Check that your script calls screen.show() after drawing. White text on a white background is also easy to miss. Start with this high-contrast example.

Example script / program

```
screen.clear("black")
screen.text(10,10,"Visible","white")
screen.show()
```

What to expect: You should see Visible. If this works, add your own drawing commands back one at a time.

C CHIP-8 instruction and example reference

These entries describe LumaRPL's assembler and profiles. A Fragment demonstrates an instruction without claiming to produce a visible full program. Use chapter 16's complete program as a starting ROM and keep its profile explicit.

Start here

Run your first CHIP-8 program

Start by drawing a digit. This complete program shows 5 and waits for a keypad key before returning.

1. Press Ctrl+P > CHIP-8 / assembly. Press F3 New and name it FIRST.

2. Type the program below, one instruction per line. Press F1 Save, then F5 Back.

3. With FIRST.asm selected, choose F4 Actions > Compatibility profile > SUPER-CHIP modern.

4. Press F2 Run. This profile supports the EXIT instruction at the end.

Example script / program

```
CLS
LD V0, 5
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: You should see 5 near the upper-left of the CHIP-8 display. Press and release X to return.

How to use these examples

Begin with Run your first CHIP-8 program. Its picture and key wait make the result visible. Instruction reference pages show smaller fragments so you can see one operation at a time.

Fragment

```
LD V0, 7
ADD V0, 2
```

What to expect: In this fragment V0 becomes 9. It does not display a number and is not a complete ROM. Add fragments to a complete program before running them.

Save and reopen your assembly

Keep the .asm file: it is the text you can edit. Running assembles a separate .ch8 ROM. Opening a .ch8 file does not turn it back into editable assembly.

1. Create an assembly file named SAVETEST and enter the program.

2. Press F1 Save, then F5 Back.

3. Select SAVETEST.asm and press F1 Open. F2 saves and builds/runs it.

Example script / program

```
CLS
LD V0, 5
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: After reopening, the same source is there. Running it in SUPER-CHIP modern displays 5 and waits for a keypad key.

Write assembly

Store and change a number

Use V0 through VF to keep small numbers. Each holds 0 to 255; arithmetic wraps around when it goes past that range. VF also carries arithmetic and drawing flags. I holds a memory address.

Fragment

```
LD V0, 0x0A
ADD V0, 2
```

What to expect: V0 becomes 12.

This instruction fragment changes registers; it does not print their values. Use Run your first CHIP-8 program for a complete visible example.

Name a place in your program

Give a jump target a label ending in a colon. Keep its spelling the same wherever you use it. Add a semicolon comment to explain a line. Instructions ignore case; labels do not.

Fragment

```
JP start
start:
LD V0, 7 ; seven
```

What to expect: Jumps to start and sets V0 to 7.

Draw sprites

Choose the digit to draw

Point I at the built-in font with LD F,V0. The next DRW reads five rows of that glyph. Change the first number below to choose another hexadecimal digit, from 0 to 15.

1. Create or open an .asm file and select SUPER-CHIP modern.

2. Type the example and press F2 Save & run.

Example script / program

```
CLS
LD V0, 9
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: This complete SUPER-CHIP program displays 9. Press and release a keypad key to return.

Work with fonts and digits

LD F,Vx chooses the small font glyph for a hexadecimal digit. For a decimal number with several digits, LD B,Vx writes its hundreds, tens and ones to three memory bytes. The fragment below splits 123 into 1, 2 and 3.

Fragment

```
LD V0, 123
LD I, 0x300
LD B, V0
```

What to expect: Memory 0x300..0x302 contains 1, 2, 3.

Keys and timers

Use the PicoCalc keypad

CHIP-8 expects sixteen hexadecimal keys. These PicoCalc keys provide them: 1 2 3 4 = 1 2 3 C Q W E R = 4 5 6 D A S D F = 7 8 9 E Z X C V = A 0 B F LD Vx,K waits for a press and release. Esc stops a running program.

Fragment

```
LD V0, K
```

What to expect: Press and release X: V0 receives the CHIP-8 key value 0. This fragment does not draw the value.

Make a short sound

Put a duration into ST to play a tone. The sound and delay timers count down at 60 ticks per second, so 30 ticks is about half a second. This complete program waits for the delay, then exits.

1. Use SUPER-CHIP modern for this program.

2. Save the source and run it with F2.

Example script / program

```
LD V0, 30
LD DT, V0
LD ST, V0
wait:
LD V1, DT
SE V1, 0
JP wait
EXIT
```

What to expect: In SCHIP, plays for roughly half a second, waits for the delay and exits.

Instruction reference

CLS

Clear the display. Call it before drawing a new picture.

Fragment

```
CLS
```

What to expect: The display is blank.

RET

Return to the instruction after the matching CALL. A RET without a call causes a return-stack error.

Fragment

```
CALL twice
JP done
twice:
ADD V0, V0
RET
done:
JP done
```

What to expect: Calls a subroutine, doubles V0 and returns. The final jump keeps it from falling into subroutine data.

JP

Jump to an address or label. A jump back to the same loop repeats until Esc stops the program.

Fragment

```
loop:
LD V0, 5
JP loop
```

What to expect: Repeatedly sets V0 to 5. Esc stops the program.

CALL

Run a subroutine at a label, then use RET to come back. Calls can nest up to sixteen levels.

Fragment

```
LD V0, 3
CALL twice
JP done
twice:
ADD V0, V0
RET
done:
JP done
```

What to expect: V0 becomes 6, then waits in the done loop.

JPOFF

Jump to the given address plus V0. Profile differences matter when you are adapting a ROM.

Fragment

```
LD V0, 2
JPOFF 0x204
LD V1, 1
LD V1, 9
```

What to expect: Jumps to 0x206, skipping the instruction that sets V1 to 1.

LD

Copy a value into a register, or move a value to or from a special register. V0 through VF hold bytes; I holds an address.

Fragment

```
LD V0, 12
LD V1, V0
```

What to expect: Both V0 and V1 contain 12.

ADD

Add a byte or another register. ADD Vx,byte wraps without changing VF; ADD Vx,Vy sets VF when there is a carry. ADD I,Vx changes the address in I.

Fragment

```
LD V0, 250
LD V1, 10
ADD V0, V1
```

What to expect: V0 becomes 4 and VF becomes 1.

SUB

Subtract Vy from Vx. A result below zero wraps around; VF is 1 when no borrow was needed.

Fragment

```
LD V0, 9
LD V1, 4
SUB V0, V1
```

What to expect: V0 becomes 5 and VF becomes 1.

SUBN

Subtract Vx from Vy and leave the answer in Vx. VF is 1 when no borrow was needed.

Fragment

```
LD V0, 3
LD V1, 8
SUBN V0, V1
```

What to expect: V0 becomes 5 and VF becomes 1.

OR

Set a bit when it is set in either register. The answer replaces Vx.

Fragment

```
LD V0, 0x0A
LD V1, 0x05
OR V0, V1
```

What to expect: V0 becomes 15.

AND

Keep only the bits set in both registers. The answer replaces Vx.

Fragment

```
LD V0, 0x0F
LD V1, 0x03
AND V0, V1
```

What to expect: V0 becomes 3.

XOR

Toggle the bits in Vx that are set in Vy. The answer replaces Vx.

Fragment

```
LD V0, 0x0F
LD V1, 0x03
XOR V0, V1
```

What to expect: V0 becomes 12.

SHR

Shift right one bit and put the bit shifted out into VF. With one register operand, that register is both the source and destination. The two-register form depends on the profile.

Fragment

```
LD V0, 9
SHR V0
```

What to expect: V0 becomes 4 and VF becomes 1.

SHL

Shift left one bit and put the high bit into VF. The two-register form depends on the profile: VIP reads Vy; the other profiles read Vx.

Fragment

```
LD V0, 0x81
SHL V0
```

What to expect: V0 becomes 2 and VF becomes 1.

SE

Skip the next instruction when Vx equals the supplied byte or register. Otherwise continue normally.

Fragment

```
LD V0, 5
SE V0, 5
LD V1, 1
LD V2, 9
```

What to expect: Skips LD V1,1; V2 becomes 9.

SNE

Skip the next instruction when Vx differs from the supplied byte or register. Otherwise continue normally.

Fragment

```
LD V0, 5
SNE V0, 6
LD V1, 1
LD V2, 9
```

What to expect: Skips LD V1,1; V2 becomes 9.

RND

Generate a random byte, then keep only the bits allowed by the mask. A mask of 0x0F gives a value from 0 to 15.

Fragment

```
RND V0, 0x0F
```

What to expect: V0 is between 0 and 15.

DRW

Draw sprite bytes from memory at I, using Vx and Vy as the position. Drawing uses XOR: set pixels toggle. VF reports a collision. Height 1 to 15 draws that many eight-pixel rows; Super-CHIP height 0 uses a 16x16 sprite. DRAW is another spelling.

Fragment

```
CLS
LD V0, 10
LD V1, 8
LD I, dot
DRW V0, V1, 1
JP done
dot:
DB 0x80, 0
done:
JP done
```

What to expect: Draws one pixel at logical (10,8). SCHIP low-resolution mode scales logical pixels on its 128x64 backing display.

SKP

Skip the next instruction if the hexadecimal key named by Vx is held. See Use the PicoCalc keypad for the physical keys.

Fragment

```
LD V0, 1
SKP V0
JP 0x202
```

What to expect: Waits here while key 1 is not held; pressing 1 skips the jump.

SKNP

Skip the next instruction if the hexadecimal key named by Vx is not held.

Fragment

```
LD V0, 1
SKNP V0
JP 0x202
```

What to expect: Skips the jump when key 1 is released.

SAVE

Store V0 through Vx in memory starting at I. This is CHIP-8 memory, not an SD save. Reset I explicitly before a later load if you want code to work across profiles.

Fragment

```
LD I, 0x300
LD V0, 7
LD V1, 9
SAVE V1
```

What to expect: Memory 0x300 contains 7 and 0x301 contains 9.

LOAD

Read V0 through Vx from memory starting at I. LD Vx,[I] is another spelling. Reset I between a SAVE and LOAD for portable code.

Fragment

```
LD I, 0x300
LD V0, 7
SAVE V0
LD I, 0x300
LD V0, 0
LOAD V0
```

What to expect: V0 returns to 7.

DB

Put bytes into the program, usually for a sprite. Jump over the data so the machine does not try to execute it. Pad an odd byte count before the next instruction.

Fragment

```
JP start
image:
DB 0xF0, 0x90, 0xF0, 0
start:
LD I, image
```

What to expect: Defines three image rows and one padding byte; I points to them.

DW

Put a 16-bit word into the program, high byte first. Prefer instruction names when writing code; words are useful when you need exact bytes.

Fragment

```
DW 0x6007
```

What to expect: Emits bytes 0x60,0x07: the opcode for LD V0,7.

EXIT

Stop the program and return to the calculator. Choose a Super-CHIP profile; classic CHIP-8 does not have this instruction.

Fragment

```
LD V0, 7
EXIT
```

What to expect: Returns after setting V0 to 7.

HIGH

Choose the Super-CHIP 128x64 display and clear it. Use a Super-CHIP profile.

Fragment

```
HIGH
CLS
```

What to expect: Selects the higher-resolution display.

LOW

Choose the classic 64x32 logical display and clear it. This mode-switch instruction needs a Super-CHIP profile.

Fragment

```
LOW
CLS
```

What to expect: Selects the classic logical display.

SCR

Scroll the display right. In high resolution it moves four pixels; low-resolution behaviour depends on the Super-CHIP profile.

Fragment

```
HIGH
SCR
```

What to expect: Moves existing high-resolution pixels right by four.

SCL

Scroll the display left. In high resolution it moves four pixels; low-resolution behaviour depends on the Super-CHIP profile.

Fragment

```
HIGH
SCL
```

What to expect: Moves existing high-resolution pixels left by four.

SCD

Scroll down by a number from 0 to 15. Use a Super-CHIP profile. Modern and HP48 profiles scale low-resolution scrolling differently.

Fragment

```
HIGH
SCD 2
```

What to expect: Moves high-resolution pixels down two rows.

FLAGSAVE

Keep V0 through Vx in the temporary Super-CHIP flag registers. Choose V0 through V7. These are not saved SD files.

Fragment

```
LD V0, 42
FLAGSAVE V0
LD V0, 0
FLAGLOAD V0
```

What to expect: V0 returns to 42.

FLAGLOAD

Restore V0 through Vx from the temporary Super-CHIP flags. Choose V0 through V7.

Fragment

```
LD V0, 42
FLAGSAVE V0
LD V0, 0
FLAGLOAD V0
```

What to expect: V0 returns to 42.

Solve a problem

Fix an assembly error

Read the line number in the assembler message and check that line. For example, LD V0,256 is out of range: a byte must be 0 to 255. Correct it to the example below, then save and build again.

1. Return to the editor and use F4 Tools > Go to line if needed.

2. Fix the instruction, then press F2 Save & run.

Example script / program

```
LD V0, 7
EXIT
```

What to expect: In SUPER-CHIP modern the corrected program builds and returns. It changes V0 without drawing anything.

A program shows a blank screen

Loading registers does not draw them. A visible picture needs I to point at sprite data and DRW to draw it. Check the selected profile as well. Try this complete program first.

Example script / program

```
CLS
LD V0, 5
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: You should see 5. If it appears, compare your own font/sprite address and draw instruction with this example. Press and release X to return.

A ROM behaves differently

If a ROM draws or responds differently than expected, check its profile. From the assembly/ROM list, choose F4 Actions > Compatibility profile. Match the ROM first; changing its code should come later.

Fragment

```
LD V0, 8
LD V1, 2
SHR V0, V1
```

What to expect: VIP shifts V1 into V0, giving 1. SCHIP shifts V0, giving 4. VF receives the bit shifted out.

CHIP-8 (VIP), CHIP-48 (HP48), SUPER-CHIP modern and SUPER-CHIP HP48 differ in shifts, memory transfers and drawing. Use SUPER-CHIP modern for the complete programs in this guide. Their EXIT instruction is not part of classic CHIP-8.

D Practice answers

RPL checks

The hypotenuse program consumes 3 and 4 and returns 5. The local-variable version returns the same result without replacing HOME A/B. A nested quoted program remains a program object until a further EVAL. The counted loop sums 1 through 5 to 15.

Lua checks

The square function prints 144; the loop prints Correct. Changing screen.font inside the console affects later console drawing in that session. calc.eval returns text, so a formatted fraction cannot be assumed to pass directly through tonumber.

CHIP-8 checks

X is keypad value 0. DRW XORs pixels, so redrawing at the same location can erase a sprite. EXIT needs a supporting SUPER-CHIP profile. Keep .asm source even after a .ch8 ROM has been generated.

Editor checks

Open example is a separate Tools entry. Returning from it restores the parent's working source and undo history. A source-line jump moves the caret; it is not a source rewrite. Undo history is a live-session aid, not a restart-persistent file format.

Calculator command additions

`FFT` and `InvFFT` accept real or complex vectors with power-of-two lengths from 1 to 1024, subject to memory. Forward coefficients are unscaled; inverse coefficients are divided by N. Use the Signal analysis workbench when you need windowing, demeaning or sample-rate-aware amplitude plots.

Matrix factorizations work on the RPL stack: `MatrixQR` returns Q R; `LQ` returns L Q; `Cholesky` returns L; `SVD` returns U s V; `Schur` returns Q T. The thin SVD uses A = U diag(s) transpose(V); Schur uses A = Q T transpose(Q). Inputs are real matrices with dimensions up to 32, subject to memory. `SVL`, `CondNum`, `SpecRad`, `ToDiagonal` and `FromDiagonal` return one result. `RREFMOD` uses a prime modulus set by `CASModulus` and preserves exact integer arithmetic.

`Normal`, `Student`, `ChiSquare` and `FDistribution` return lower-tail probabilities. Their `Inv` counterparts take a probability strictly between zero and one. See the command reference for argument order.

`Taylor` takes expression, variable, point and order; `FiniteDifference` takes expression, variable and step. `Limit` takes expression, variable and finite real point, with support for analytic expressions and removable zero-over-zero singularities. Taylor and Limit use radians for analytic trigonometric calculations. `DerivX` and `IntVX` use the selected algebra variable. Variables in these command arguments stay symbolic even when a stored variable has the same name.

Calculator shorthand conversion does not run inside `INPUT` prompts. Validators receive exactly the text that was entered, including parentheses, `^`, `!`, `?` and ordinary program variable names.

E Credits and further reading

LumaRPL design direction and device testing: **Iain Bennett**. Cover photograph supplied by Iain Bennett. The following components and people make the calculator and its development possible.

The combined PicoCalc port is distributed under GPL-3.0. Components and fonts retain their own notices and license terms. The summary below does not replace the complete license texts shipped with the corresponding source and license bundle.

DB48X

Christophe de Dinechin and contributors. RPL engine, arithmetic, symbolic mathematics, calculator fonts and upstream simulator. LGPL-3.0 terms supplied upstream.

https://github.com/c3d/db48x

ClockworkPi PicoCalc

ClockworkPi and contributors. Hardware, published board design and driver origins. Hardware/reference; driver notices retained.

https://github.com/clockworkpi/PicoCalc

UF2 Loader 2.4.1

pelrun and contributors. Loader workflow, display/keyboard/SD driver origins. GPL-3.0 driver/source terms.

https://github.com/pelrun/uf2loader

Raspberry Pi Pico SDK 2.2.0

Raspberry Pi and contributors. RP2350 platform and runtime integration. BSD-3-Clause and component-specific notices.

https://github.com/raspberrypi/pico-sdk

FatFS R0.16

ChaN. SD filesystem. Permissive notice retained in sources.

https://elm-chan.org/fsw/ff/

SwissMicros DMCP

SwissMicros. Interface declarations used by the port. BSD-3-Clause declarations; no ROM linked.

https://github.com/swissmicros/DMCP_SDK

Lua 5.4.9

Lua.org, PUC-Rio. Embedded scripting runtime. MIT; Copyright 1994-2026 Lua.org, PUC-Rio.

https://www.lua.org/

CYW43 driver / firmware

George Robotics / Raspberry Pi and contributors. Pico 2 W radio support. Raspberry Pi semiconductor-license terms; original notices retained.

https://github.com/georgerobotics/cyw43-driver

lwIP

lwIP authors and contributors. Network stack. BSD-style notice.

https://savannah.nongnu.org/projects/lwip/

Mbed TLS 3.6.7

Mbed TLS authors and contributors. TLS and certificate validation. Apache-2.0 OR GPL-2.0-or-later, as supplied.

https://github.com/Mbed-TLS/mbedtls

Google Trust Services

Google Trust Services. Public GTS Root R4 trust certificate. Certificate source and DER fingerprint retained.

https://pki.goog/repository/

Frankfurter

Frankfurter contributors and data providers. Exchange-rate reference observations; underlying official providers retain their attribution. External data service; cached observation dates retained.

https://frankfurter.dev/

Pixel Operator Mono

Jayvee Enaguas / GrandChaos9000; The Grandoplex Project. Compact source-editor font base. SIL Open Font License 1.1; Copyright 2009-2015 The Grandoplex Project.

https://github.com/c3d/db48x/tree/master/fonts

FogSans

Intel Corporation; DB48X font modifications by its contributors. Calculator/compact font source and supplemental editor glyphs. Apache-2.0 font software; Copyright 2012 Intel Corporation.

https://github.com/c3d/db48x/tree/master/fonts

CMSIS

Arm and contributors. Arm platform headers. Original component notices in source/licenses.

https://github.com/ARM-software/CMSIS_6

Pico printf

Marco Paland and contributors. SDK formatted-output component; this build selects compiler/Newlib printf. MIT notice retained.

https://github.com/mpaland/printf

GNU Arm toolchain, Newlib and C++ runtime

Their authors and contributors. Compilation and embedded runtime support. Original licenses and runtime exceptions retained.

https://developer.arm.com/Tools%20and%20Software/GNU%20Toolchain

picotool

Raspberry Pi and contributors. Firmware build and UF2 inspection. Build/validation tool, not a calculator application.

https://github.com/raspberrypi/picotool

Timendus CHIP-8 test suite

Timendus and contributors. VM validation only. GPL-3.0 test source; ordinary SD bundle omits test ROMs.

https://github.com/Timendus/chip8-test-suite

Qt and QEMU

Their authors and contributors. Desktop simulator, host rendering and Arm validation. Development/testing tools; not linked into the calculator firmware.

https://www.qt.io/

DejaVu fonts

Bitstream, Tavmjong Bah and DejaVu contributors. Embedded PDF typography only. Font-specific licenses reproduced in the manuals and PDF-FONT-NOTICES.txt.

https://dejavu-fonts.github.io/

ReportLab, Poppler, Pillow, pypdf and pdfplumber

Their authors and contributors. Manual generation, photo orientation, PDF structure, geometry and rendering checks only. Documentation build tools; not linked into calculator firmware.

https://www.reportlab.com/

Read next

DB48X source and documentation

Lua 5.4 reference manual

PicoCalc hardware and keyboard resources

UF2 Loader source and workflow

Frankfurter API and provider information

HP 48G Series User's Guide, Hewlett-Packard

HP 48G Series Advanced User's Reference, Hewlett-Packard

The HP books are historical documentation-design inspiration and further reading. The prose, procedures and examples here were prepared for LumaRPL; use the PicoCalc controls and current LumaRPL reference when following this book.

F PDF font license notices

These notices apply to the DejaVu fonts embedded in the PDF, not to the PicoCalc firmware. The complete text is retained in the manual source as PDF-FONT-NOTICES.txt.

DejaVu Sans and DejaVu Sans Mono Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Copyright (c) 2006 by Tavmjong Bah. All Rights Reserved. DejaVu changes are in public domain

Fonts are (c) Bitstream (see below). DejaVu changes are in public domain. Glyphs imported from Arev fonts are (c) Tavmjung Bah (see below)

Bitstream Vera Fonts Copyright ----------

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Bitstream" or the word "Vera".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the "Bitstream Vera" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc., shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.

Arev Fonts Copyright ----------

Copyright (c) 2006 by Tavmjong Bah. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the modifications to the Bitstream Vera Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Tavmjong Bah" or the word "Arev".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the "Tavmjong Bah Arev" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL TAVMJONG BAH BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the name of Tavmjong Bah shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from Tavmjong Bah. For further information, contact: tavmjong @ free . fr.

http://dejavu.sourceforge.net/wiki/index.php/License

Index

Alphabetical index of section subjects. The Contents lists chapter starts.

A

A blank screen . . . 35
A build error . . . 35
A counted loop with a name . . . 15
A first picture . . . 24
A moving dot . . . 25
A saddle surface . . . 26
A small diagnosis exercise . . . 19
A three-language restart test . . . 38
ADD . . . 62
AND . . . 63
Ask the calculator for one expression . . . 22
Available libraries . . . 22

C

calc.eval . . . 44
CALL . . . 61
Catch an expected failure . . . 29
CHIP-8 checks . . . 71
Choose a branch . . . 15
CLS . . . 60
CMSIS . . . 75
Comments and readable source . . . 13
Compare sine and cosine . . . 26
Create, save and reopen . . . 7

D

DB 67
DB48X 73
Degrees and radians 23
Deliberately return a program 14
Display modes 33
Display settings inside examples 17
Draw a saved colour sprite 27
Draw on screen 45
Draw sprites 58
Draw the digit 5 30
Draw your own eight-pixel-wide image 31
DRW 65
DW 67

E

Editor checks 71
EXIT 67

F

Find the actual line 29
FLAGLOAD 69
FLAGSAVE 69

G

Give inputs local names 14

H

HIGH 67

I

Insert a command template 9
Instruction reference 60

J

JP 61
JPOFF 61

K

Keep work and execution separate 37
keyboard.down 52
keyboard.poll 51
keyboard.wait 52
Keys and timers 59

L

Labels and data 31
Language reference 9
LD 62
Lists and arrays 17
LOAD 66
LOW 68
Lua checks 71

M

Measure before placing text 25

O

Objects and mathematics 40
Open example is a separate choice 9
OR 63

P

Paint a reusable sprite 35
Parse errors and runtime errors 19

Play a short tone 27
Plot graphs 49
plot.clear 49
plot.function2d 50
plot.show 51
plot.surface 50
plot.window 50
Prefer a native plot for mathematics 37
Preserve physical quantities 17
Protect existing work 6

R

Read a key without blocking 27
Read and run examples 5
Read next 76
Registers and numbers 31
Repeat and choose 20
RET 60
RND 65
RPL checks 71

S

SAVE 66
SCD 68
SCL 68
SCR 68
screen.circle 47
screen.clear 45
screen.font 47
screen.line 46
screen.measure 48

screen.pixel . . . 46
screen.rect . . . 46
screen.show . . . 48
screen.sprite . . . 48
screen.text . . . 47
SE . . . 64
Select the expected interpreter behaviour . . . 33
Settings and structure . . . 41
SHL . . . 64
SHR . . . 64
SKNP . . . 66
SKP . . . 65
SNE . . . 64
Solve a problem . . . 54, 69
sound.beep . . . 53
sound.stop . . . 53
Source and ROM are different files . . . 30
Square a number . . . 10
Stack and names . . . 39
Start here . . . 42, 56
Start with an observable result . . . 37
Stay within the budget . . . 29
Step through a program . . . 19
Store a value or program . . . 14
SUB . . . 62
SUBN . . . 63
system.millis . . . 53
system.sleep . . . 52

T

Tables and strings . . . 22

Test before or after the body 16
Test the saved copy 11
Text controls 8
The same program on SD 10
The sixteen-key mapping 33
Timers 32
Treat boundaries as part of the design 38
Two inputs: a hypotenuse 12

U

Unsaved changes 8
Use an appropriate font 24
Use keys and sound 51
Use the full reference 18
Use the Lua console 21
Useful stack operations 12

V

Values and functions 20

W

Write assembly 58
Write Lua 43

X

XOR 63